SEGURANÇA DA INFORMAÇÃO · 2ª AVALIAÇÃO

Criptografia assimétrica com RSA-OAEP

Implementação do RSA do zero, em Python puro: geração de chaves, cifragem e decifragem de qualquer mensagem.

Equipe: Thales Henrique · (integrante 2) · (integrante 3) · (integrante 4)
Novembro de 2026

MOTIVAÇÃO

Como combinar um segredo sem se encontrar antes?

Criptografia simétrica

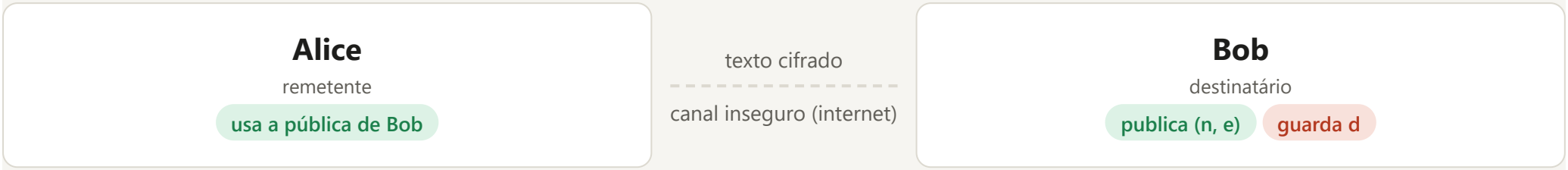
Uma única chave serve para cifrar e para decifrar (ex.: AES).

Problema: a chave precisa chegar ao destinatário por um canal seguro. Se alguém a intercepta, lê tudo.

Criptografia assimétrica

São duas chaves matematicamente ligadas:

- **pública**: cifra. Pode ser divulgada a todos.
- **privada**: decifra. Fica só com o dono.



Quem intercepta o canal vê a chave pública e o texto cifrado, mas sem a chave privada não consegue decifrar.

O ALGORITMO

RSA (Rivest, Shamir e Adleman, 1977)

Geração das chaves

1. Sortear dois primos grandes **p** e **q** (1024 bits cada)
2. $$n = p \cdot q$$
3. $$\lambda(n) = \text{mmc}(p-1, q-1)$$
4. Escolher **e = 65537** com mdc(e, λ(n)) = 1
5. $$d = e^{-1} \bmod \lambda(n)$$

Chave **pública: (n, e)**. Chave **privada: d**.

Cifrar (chave pública)

$$c = m^e \bmod n$$

Decifrar (chave privada)

$$m = c^d \bmod n$$

Por que é seguro?

Como $e \cdot d \equiv 1 \pmod{\lambda(n)}$, temos $(m^e)^d \equiv m \pmod{n}$. Calcular **d** exige **λ(n)**, e calcular λ(n) exige **fatorar n**. Para um n de 2048 bits, não há algoritmo conhecido que faça isso em tempo viável.

EXEMPLO COM NÚMEROS PEQUENOS

p = 61, q = 53, e = 17

$n = 61 \cdot 53$	**3233**
$\lambda(n) = \text{mmc}(60, 52)$	**780**
e	**17**
$d = 17^{-1} \bmod 780$	**413** (17 · 413 = 7021 = 9 · 780 + 1)

Mensagem m = 65 (letra “A” em ASCII)

$c = 65^{17} \bmod 3233 = 2790$

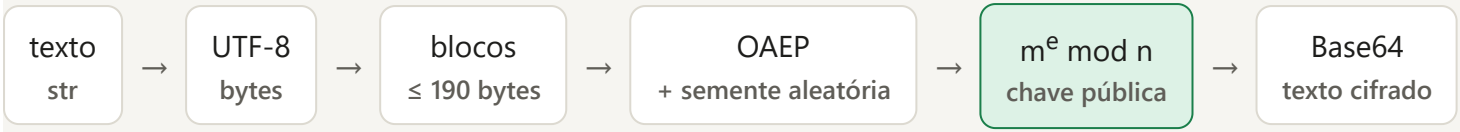
$m = 2790^{413} \bmod 3233 = 65$ ✓

Esse mesmo exemplo é verificado nos testes automatizados (`TestExemploLivroTexto`), que também conferem a ida e volta de **todas** as 3233 mensagens possíveis.

No programa real, p e q têm 1024 bits (cerca de 309 dígitos cada) e n tem 2048 bits.

VISÃO GERAL DO CÓDIGO

Arquitetura do projeto



Arquivo	Responsabilidade
`aritmetica.py`	Euclides estendido, inverso modular, Miller-Rabin, geração de primos
`chaves.py`	Gera, valida, salva e carrega o par de chaves
`oaep.py`	Preenchimento OAEP (SHA-256 + MGF1)
`rsa.py`	m^e, c^d com CRT e blinding, blocos, Base64
`main.py`	Linha de comando: `demo`, `gerar-chaves`, `cifrar`, `decifrar`
`tests/`	23 testes automatizados

Princípios

- **Sem bibliotecas de criptografia**: tudo está visível no código
- Segue a especificação oficial **RFC 8017** (PKCS #1 v2.2)
- Usa só a biblioteca padrão (`secrets` , `hashlib`)
- Cada módulo tem uma única responsabilidade

CÓDIGO 1 · ARITMETICA.PY

Gerando primos de 1024 bits

cripto_rsa/aritmetica.py

```python
def miller_rabin(n: int, rodadas: int = 40) -> bool:
    if n < 2:
        return False
    for p in PRIMOS_PEQUENOS:
        if n % p == 0:
            return n == p

    # Escreve n - 1 como d * 2^s, com d ímpar.
    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1

    for _ in range(rodadas):
        a = secrets.randbelow(n - 3) + 2  # base aleatória em [2, n-2]
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False  # "a" é testemunha de que n é composto
    return True


def gerar_primo(bits: int) -> int:
    while True:
        candidato = secrets.randbits(bits)
        candidato |= (1 << (bits - 1)) | (1 << (bits - 2)) | 1
        if miller_rabin(candidato):
            return candidato
```

Miller-Rabin

Teste probabilístico: um primo sempre passa. Um número composto passa em uma rodada com chance de no máximo ¼. Com **40 rodadas**, a chance de erro fica abaixo de $4^{-40} \approx 10^{-24}$.

Filtro rápido

Antes do teste, o candidato é dividido pelos primos menores que 2000 (crivo de Eratóstenes). Isso descarta a maioria dos compostos quase de graça.

Bits fixados

Os 2 bits mais altos ficam em 1, o que garante que n = p·q tenha exatamente 2048 bits. O bit mais baixo fica em 1, o que garante um candidato ímpar. O módulo `secrets` usa o gerador seguro do sistema operacional.

CÓDIGO 2 · CHAVES.PY

Montando o par de chaves

cripto_rsa/chaves.py

```python
def construir_chave_privada(p, q, e=E_PADRAO) -> ChavePrivada:
    if p == q:
        raise ValueError("p e q devem ser primos distintos")
    lambda_n = mmc(p - 1, q - 1)
    if mdc(e, lambda_n) != 1:
        raise ValueError("e não é coprimo com λ(n)")
    d = inverso_modular(e, lambda_n)
    return ChavePrivada(
        n=p * q,
        e=e,
        d=d,
        p=p,
        q=q,
        dp=d % (p - 1),
        dq=d % (q - 1),
        qinv=inverso_modular(q, p),
    )


def gerar_par_de_chaves(bits=2048, e=E_PADRAO):
    ...
    while True:
        p = gerar_primo(bits_p)
        q = gerar_primo(bits_q)
        # p e q muito próximos permitem fatorar n pelo método de Fermat.
        if abs(p - q).bit_length() <= bits_p - 100:
            continue
        ...
```

Inverso modular

`inverso_modular` usa o **algoritmo de Euclides estendido**: encontra x e y com $e \cdot x + \lambda \cdot y = 1$, e então $d = x \bmod \lambda$.

dp, dq, qinv

São valores pré-calculados para o **Teorema Chinês do Resto**, que deixa a decifragem cerca de 3 vezes mais rápida (próximos slides).

Cuidados extras

- `repr=False` nos campos secretos: `print(chave)` não mostra d, p nem q
- `validar()` confere a matemática ao carregar a chave de um arquivo
- `.gitignore` impede que a chave privada vá para o GitHub

PREENCHIMENTO (PADDING)

Por que não usar o RSA “puro”?

Determinístico

A mesma mensagem sempre gera o mesmo texto cifrado. Um atacante cifra “SIM” e “NÃO” com a chave pública e compara com o que interceptou.

Maleável

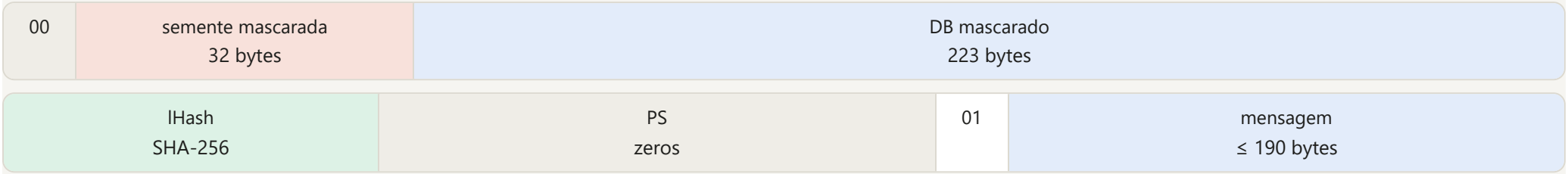
Multiplicar c por 2e faz o destinatário decifrar **2·m**, e ninguém percebe a alteração.

Mensagens curtas

Se $m^e < n$, o “mod n” não tem efeito, e basta tirar a raiz e-ésima de c para recuperar m.

A solução: OAEP (Bellare e Rogaway, 1994)

Antes de elevar a e, a mensagem é misturada com uma **semente aleatória** de 32 bytes e recebe um hash de verificação. Bloco de k = 256 bytes:



Os 190 bytes vêm de 256 − 2·32 − 2. A semente mascara o DB, e o DB mascara a semente (MGF1 com SHA-256).

CÓDIGO 3 · OAEP.PY

Codificação OAEP (RFC 8017 §7.1)

cripto_rsa/oaep.py

```python
def mgf1(semente: bytes, tamanho: int) -> bytes:
    saida = bytearray()
    contador = 0
    while len(saida) < tamanho:
        saida += HASH(semente + contador.to_bytes(4, "big")).digest()
        contador += 1
    return bytes(saida[:tamanho])


def codificar(mensagem: bytes, k: int, rotulo: bytes = b"") -> bytes:
    if len(mensagem) > tamanho_maximo_mensagem(k):
        raise ValueError("mensagem grande demais para um bloco OAEP")

    l_hash = HASH(rotulo).digest()
    ps = b"\x00" * (tamanho_maximo_mensagem(k) - len(mensagem))
    db = l_hash + ps + b"\x01" + mensagem

    semente = secrets.token_bytes(TAMANHO_HASH)
    db_mascarado = _xor(db, mgf1(semente, k - TAMANHO_HASH - 1))
    semente_mascarada = _xor(semente, mgf1(db_mascarado, TAMANHO_HASH))

    return b"\x00" + semente_mascarada + db_mascarado
```

MGF1

"Estica" um valor curto até o tamanho desejado, concatenando SHA-256(semente || 0), SHA-256(semente || 1) e assim por diante.

Decodificação

Desfaz os dois XORs e confere: o 1º byte é 00, o lHash bate, o PS só tem zeros e existe o separador 01.

Erro genérico

Qualquer falha gera o mesmo `ErroDecifragem("falha na decifragem")`. Se o programa dissesse *qual* verificação falhou, um atacante poderia usar isso para decifrar a mensagem (ataque de Manger, 2001).

CÓDIGO 4 · RSA.PY

Cifragem e decifragem

cripto_rsa/rsa.py

```python
def rsaep(chave: ChavePublica, m: int) -> int:
    """Primitiva de cifragem: c = m^e mod n."""
    return pow(m, chave.e, chave.n)


def _rsadp_crt(chave: ChavePrivada, c: int) -> int:
    m1 = pow(c, chave.dp, chave.p)
    m2 = pow(c, chave.dq, chave.q)
    h = (chave.qinv * (m1 - m2)) % chave.p
    return m2 + h * chave.q


def rsadp(chave: ChavePrivada, c: int) -> int:
    """m = c^d mod n, com blinding."""
    while True:
        r = secrets.randbelow(chave.n - 2) + 2
        if mdc(r, chave.n) == 1:
            break
    c_cego = (c * pow(r, chave.e, chave.n)) % chave.n
    m_cego = _rsadp_crt(chave, c_cego)
    return (m_cego * inverso_modular(r, chave.n)) % chave.n


def cifrar_bloco(chave: ChavePublica, bloco: bytes) -> bytes:
    k = chave.tamanho_bytes
    em = oaep.codificar(bloco, k)
    c = rsaep(chave, bytes_para_int(em))
    return int_para_bytes(c, k)
```

pow(m, e, n)

O `pow` do Python faz **exponenciação modular rápida** (quadrados sucessivos): cerca de 2048 multiplicações em vez de 65537.

CRT (Teorema Chinês do Resto)

Em vez de uma exponenciação módulo n (2048 bits), o código faz duas módulo p e q (1024 bits cada) e junta os resultados com a fórmula de Garner. É cerca de 3 vezes mais rápido.

Blinding

c é multiplicado por r^e, com r aleatório, antes de usar a chave privada, e o fator r é removido no final. O tempo de cálculo deixa de depender do c que o atacante escolheu, o que protege contra **ataques de temporização**.

CÓDIGO 5 · RSA.PY

Mensagens de qualquer tamanho

cripto_rsa/rsa.py

```python
def cifrar(chave: ChavePublica, mensagem: bytes) -> bytes:
    k = chave.tamanho_bytes
    limite = oaep.tamanho_maximo_mensagem(k)
    pedacos = [mensagem[i : i + limite] for i in range(0, len(mensagem), limite)]
    if not pedacos:
        pedacos = [b""]  # mensagem vazia também vira um bloco válido
    return b"".join(cifrar_bloco(chave, p) for p in pedacos)


def decifrar(chave: ChavePrivada, cifrado: bytes) -> bytes:
    k = chave.tamanho_bytes
    if not cifrado or len(cifrado) % k != 0:
        raise ErroDecifragem("falha na decifragem")
    blocos = [cifrado[i : i + k] for i in range(0, len(cifrado), k)]
    return b"".join(decifrar_bloco(chave, b) for b in blocos)


def cifrar_texto(chave: ChavePublica, texto: str) -> str:
    return base64.b64encode(cifrar(chave, texto.encode("utf-8"))).decode("ascii")
```

Divisão em blocos

Um bloco de 2048 bits comporta no máximo **190 bytes**. Uma mensagem de 1000 bytes vira 6 blocos cifrados de 256 bytes cada.

Texto: UTF-8 e Base64

O texto é convertido em bytes UTF-8, o que suporta acentos e emoji. O resultado é codificado em Base64 para poder ser copiado e colado como texto.

No mundo real

Usa-se **criptografia híbrida**: o RSA cifra só uma chave AES aleatória, e o AES cifra os dados (como no TLS e no PGP). Aqui usamos o RSA diretamente porque esse é o objetivo do trabalho.

FUNCIONAMENTO

Executando o programa

```
$ python main.py demo -m "Olá, turma! Mensagem secreta 🔐"

═══ 1. Gerando par de chaves RSA de 2048 bits ═══════════════
Tempo: 0.40 s
n (módulo, público) = c9a6519bb1b9450ec91bd04874a2a484415f007f ... c322e9960dd06c6de3702acaf00b23784b2660b9
e (expoente público) = 65537
d (expoente privado) = 3e137c45f0c537142126f5cc8853554ffd8bd06c ... 26fc7029b8c837034b0e1c875408975b1cfc474d  [SECRETO]

═══ 2. Mensagem original ════════════════════════════════════
Olá, turma! Mensagem secreta 🔐
34 bytes em UTF-8 (cada bloco OAEP comporta até 190 bytes)

═══ 3. Cifrando com a chave PÚBLICA ═════════════════════════
1 bloco(s) de 256 bytes
Base64: PSGRh4mxa/wj5d5uh/Bc3e2V7mfyqkOEuhDIK8nK ... srWEiE18SFWz+2K2SvN4KPsTuUdrehau8jYsRA==
Cifrando de novo a MESMA mensagem: CYr0zJLTVKD2/kO5xLpF ... nRVTtPjFdgtCi025CA==
Os textos cifrados são diferentes? True  (OAEP é aleatório)

═══ 4. Decifrando com a chave PRIVADA ═══════════════════════
Olá, turma! Mensagem secreta 🔐

═══ 5. Verificação ══════════════════════════════════════════
SUCESSO: a mensagem decifrada é idêntica à original.
Texto cifrado com 1 bit alterado foi REJEITADO, como esperado.
```

Uso separado (Alice e Bob)

```
python main.py gerar-chaves --bits 2048 --pasta chaves
python main.py cifrar   --chave chaves/publica.json -m "Encontro às 15h" -s cifrado.txt
python main.py decifrar --chave chaves/privada.json -a cifrado.txt
```

QUALIDADE

23 testes automatizados

Grupo	O que é verificado
Aritmética	Euclides estendido, inverso modular, Miller-Rabin com primos conhecidos e **números de Carmichael** (561, 1105, 8911)
Livro-texto	p = 61, q = 53: c = 2790, e a ida e volta das 3233 mensagens possíveis
Ida e volta	Mensagem vazia, 1 byte, acentos e emoji, todos os 256 bytes, 1000 bytes (vários blocos), tamanhos no limite exato do bloco
Segurança	Cifragem aleatória, texto cifrado não contém a mensagem, adulteração detectada, chave errada rejeitada, `repr` não vaza segredos
Entradas inválidas	Tamanho errado, Base64 inválido, chave menor que 1024 bits
Persistência	Salvar e carregar as chaves em JSON

```
$ python -m unittest discover -s tests
......................
----------------------------------------------------------------------
Ran 23 tests in 0.278s

OK
```

Os testes encontraram um bug

Um texto “Base64” com acentos lançava `ValueError` em vez de `ErroDecifragem`. O teste `test_entradas_invalidas` pegou o problema, e a correção foi tratar as duas exceções.

EXEMPLO VISUAL

RSA de livro-texto, byte a byte

Versão simplificada, com primos pequenos e sem OAEP, que cifra cada byte separadamente. Repare que bytes iguais geram o mesmo texto cifrado: esse é o problema que o OAEP resolve.

p (primo) 61

q (primo) 53

mensagem banana 🔐

n = 3233 e = 17 λ(n) = 780 d = 413

byte (m)	98	97	110	97	110	97	32	240	159	148	144
cifrado $c = m^e \bmod n$	2570	1632	2235	1632	2235	1632	1992	2029	3180	543	2375
decifrado $c^d \bmod n$	98	97	110	97	110	97	32	240	159	148	144

Mensagem recuperada: **banana** 🔐 ✓ idêntica · células destacadas: bytes iguais geraram cifras iguais

DECISÕES TÉCNICAS

Segurança: o que foi feito e o que falta

Implementado

- **OAEP + SHA-256**: cifragem aleatória e com verificação de integridade
- **secrets**: aleatoriedade criptograficamente segura
- Chaves de **2048 bits** (recomendação do NIST); mínimo de 1024
- **e = 65537**; p e q distantes (contra o método de Fermat)
- **Blinding** contra ataques de temporização
- **Erro genérico** contra o ataque de Manger
- Validação da chave carregada; segredos fora do `repr`

Limitações (projeto didático)

- Python puro **não garante tempo constante**. Em produção, use bibliotecas auditadas, como `cryptography` e OpenSSL.
- Não usamos criptografia híbrida (RSA + AES), que é o padrão para dados grandes.
- A chave privada é salva sem senha. O ideal seria usar PKCS#8 com cifragem.
- O programa não faz assinatura digital, então não há autenticação do remetente.

ENCERRAMENTO

Conclusão

- Implementamos o **RSA completo**: primos, chaves, cifragem e decifragem
- A mensagem original é recuperada **exatamente**, com qualquer tamanho e com acentos ou emoji
- O **OAEP** torna a cifragem aleatória e detecta adulteração
- Há **23 testes** automatizados, e todos passam
- Na criptografia, segurança depende tanto dos **detalhes de implementação** quanto da matemática

Referências

- Rivest, Shamir, Adleman: *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*, 1978
- **RFC 8017**: PKCS #1: RSA Cryptography Specifications v2.2
- Bellare, Rogaway: *Optimal Asymmetric Encryption*, 1994
- Manger: *A Chosen Ciphertext Attack on RSA OAEP*, 2001
- NIST SP 800-57: Recommendation for Key Management
- Stallings: *Criptografia e Segurança de Redes*

Código: github.com/ThalesTHM/Segunda-avalia-o-seguran-a-da-informa-o